Directed Reading: GPU Programming for Motion Planning Acceleration

Duration: 12 weeks · ~8hrs/week (readings + mini-project)

Prerequisites: C/C++ proficiency, basic computer architecture (caches, pipelines), linear algebra, familiarity with a motion planning stack (e.g., OMPL, cuRobo, or MPPI)

Hardware: Access to an NVIDIA GPU (Ampere or newer recommended; Blackwell ideal. I can provide access to GB10)

Textbook reference (might help; available on SFU library):
https://github.com/cfregly/ai-performance-engineering

Course Philosophy

Motion planning workloads—collision checking, signed-distance-field queries, sampling, trajectory optimization—are embarrassingly parallel in parts and irregular in others. This course builds GPU programming fluency from first principles, then applies it to the specific computational patterns found in motion planning: batched geometric queries, sparse graph traversals, parallel tree expansion, and real-time trajectory optimization. Each week pairs foundational GPU reading with a hands-on mini-project that gradually builds toward a capstone motion-planning accelerator.

Phase 1: GPU Architecture & CUDA Fundamentals (Weeks 1–4)

Week 1 — The GPU Execution Model

Goal: Understand the GPU as a throughput-oriented machine and write your first kernels.

Readings

1. A Brief History of GPUs — Evolution from graphics pipelines to general-purpose compute; establishes why the hardware looks the way it does.

2. GPU Glossary — Device Hardware — Reference for SM, warp, register file, and tensor core terminology. Skim the full glossary; bookmark for later.
3. CUDA C Programming Guide, Chapters 1–3 (NVIDIA docs) — Programming model (grids, blocks, threads), memory spaces, kernel launch syntax.

Supplementary

- NVIDIA GTC talk: Maximize Bandwidth and Latency — First 20 minutes cover the execution model at a high level.

Mini-Project 1 — Parallel Distance Matrix

Implement a CUDA kernel that computes the pairwise Euclidean distance matrix for N points in 3D (input: `float3 *points`, output: `N×N float *dist`). Profile with `nsys` and report achieved bandwidth vs. peak. Vary N from 1K to 64K and plot throughput.

Why this matters for motion planning: Pairwise distance computation is the core primitive in PRM/RRT neighbor searches, collision proximity queries, and nearest-neighbor graphs.

Week 2 — Memory Hierarchy: Global, Shared, Registers

Goal: Internalize the GPU memory hierarchy and learn to exploit shared memory for data reuse.

Readings

1. CUDA C Programming Guide, Chapter 5 (Performance Guidelines) — Coalescing, occupancy, shared memory usage patterns.
2. Shared Memory Microbenchmarks — Empirical measurements of bank conflicts, padding strategies, and throughput.
3. Making Matrix Transpose Really Fast on Hopper — Banking, layout, and conflict-free access patterns through the lens of a deceptively simple kernel.

Supplementary

- CUDA C Programming Guide, Appendix C (warp-level intrinsics) — Useful reference for `__shfl_*` and warp-level reductions.

Mini-Project 2 — Tiled Sphere–Sphere Collision Check

Given a batch of B robot configurations, each with J spheres (representing a simplified robot geometry), and E environment obstacle spheres, implement a tiled shared-memory kernel that checks for collisions. Return a boolean per configuration. Use shared memory to stage tiles of obstacle spheres and amortize global memory traffic.

Why this matters: Sphere-based collision checking is the workhorse of cuRobo and similar GPU motion planners. This is the inner loop you'll optimize all course.

Week 3 — Warp-Level Programming & Reductions

Goal: Move from block-level to warp-level thinking; master reductions and scans.

Readings

1. CUDA GEMM — Step by Step (Sections 1–6 only) — Not for GEMM itself yet, but for its clear exposition of tiling, register blocking, and occupancy tuning.
2. Register-Level Tiling — Algorithmic basics of how data moves from global → shared → registers.
3. Mark Harris, "Optimizing Parallel Reduction in CUDA" (NVIDIA whitepaper, classic) — The canonical walk through 7 reduction kernels from naïve to warp-shuffle.

Supplementary

- CUB library documentation (block-level primitives: `BlockReduce`, `BlockScan`, `BlockRadixSort`).

Mini-Project 3 — Parallel k-Nearest Neighbors (Brute Force)

For a set of N sample configurations in C-space (dimension d = 6–7 for a typical manipulator), implement a brute-force k-NN kernel using warp-level reductions for the per-query top-k selection. Compare against a naïve sort-based approach. Benchmark for N = 10K–500K, k = 8.

Why this matters: k-NN in C-space is the graph construction step in PRM and the neighbor-rewiring step in RRT*. Doing it on-GPU eliminates a major CPU–GPU synchronization point.

Week 4 — Streams, Concurrency & Profiling

Goal: Learn to overlap compute, memory transfers, and kernel launches; become proficient with NVIDIA profiling tools.

Readings

1. CUDA C Programming Guide, Chapter 3.2.8 (Asynchronous Concurrent Execution) — Streams, events, default stream behavior.
2. CUDA Graphs — Accelerating Generative AI — How CUDA graphs eliminate launch overhead for repeated workloads.
3. Nsight Systems and Nsight Compute documentation (quick-start guides) — Learn to read a timeline, identify stalls, and interpret occupancy/warp schedulers.

Supplementary

- NVIDIA best-practices guide: "Asynchronous Data Transfers" section.

Mini-Project 4 — Pipelined Collision-Check + Path-Scoring

Extend your Week 2 collision checker: given a batch of candidate trajectories (sequences of configurations), use CUDA streams to overlap (a) collision checking of trajectory batch i with (b) cost-function scoring of the already-checked batch i–1 and (c) async memcpy of results for batch i–2 back to the host. Profile with `nsys` to show overlapped execution. Then capture the pipeline as a CUDA graph and measure launch-overhead reduction.

Why this matters: Real-time motion planning (MPPI, STORM) evaluates thousands of candidate rollouts per control cycle. Pipelining and graph capture are how you hit 100 Hz+ replanning rates.

Phase 2: Intermediate GPU Optimization (Weeks 5–8)

Week 5 — Dense Linear Algebra on GPU: GEMM Deep Dive

Goal: Understand GEMM as the canonical GPU kernel; learn the tiling and data-movement hierarchy that applies to all compute-bound GPU code.

Readings

1. CUDA GEMM — Step by Step (complete) — Full walk-through from naïve to near-cuBLAS performance.
2. Outperforming cuBLAS on H100 — Real engineering worklog of pushing GEMM on Hopper; exposes the gap between textbook and production.
3. CuTE Layouts — CUTLASS's layout abstraction; how logical tile coordinates map to physical memory.

Supplementary

- CUTLASS documentation: GemmUniversal interface and tiled-GEMM concepts.

Mini-Project 5 — Batched Small-Matrix Multiply for FK/Jacobians

Implement a custom batched GEMM kernel for 4×4 homogeneous transforms (forward kinematics chain) for B configurations simultaneously. Compare against cuBLAS `cublasSgemmBatched` for these small matrix sizes. Measure where your custom kernel wins (small M/N/K) vs. where cuBLAS wins.

Why this matters: Forward kinematics and Jacobian computation are chains of small matmuls. cuBLAS has high overhead for tiny matrices; custom kernels with register-level storage dominate.

Week 6 — Tensor Cores & MMA Programming

Goal: Understand hardware matrix-multiply units and how to program them at the PTX and WMMA levels.

Readings

1. Tensor Core Evolution: Volta to Blackwell — Architecture evolution, data formats (FP16, TF32, FP8, INT8), and throughput scaling.
2. WMMA and MMA Programming + MMA PTX Notes — Hands-on code for warp-level matrix operations.
3. Fast GEMM with Tensor Cores — From-scratch implementation using tensor cores.

Mini-Project 6 — Tensor-Core Batched Jacobian Transpose Multiply

For a manipulator Jacobian J (6×d, d = 6–7), implement the product $J^T \cdot f$ (force/torque mapping) for a batch of B configurations using WMMA intrinsics (pad to 16×16 tiles). Compare FP32, TF32, and FP16 accuracy for motion-planning-relevant precision requirements. Determine whether tensor cores offer a speedup for these small problem sizes.

Why this matters: Evaluating whether tensor cores help at motion-planning matrix sizes (small, batched) is an open practical question. The answer informs architecture decisions for real-time planners.

Week 7 — Advanced Memory: TMA & Async Copy

Goal: Learn Hopper/Blackwell's Tensor Memory Accelerator and asynchronous bulk copy for decoupling compute from data movement.

Readings

1. TMA Basics with CuTE — How TMA descriptors work, address generation, and multicast.
2. Blackwell GEMM with CUTLASS — Tensor Memory (TMEM) and 5th-generation tensor cores.
3. Persistent Dynamic Launch (PDL) — Dynamic kernel scheduling for persistent kernels; important for workloads with variable parallelism.

Supplementary

- CUTLASS 3.x source: `cute/arch/copy_sm90_tma.hpp` — Read the TMA copy atoms.
- Code repository: Yang-YiFan blog code

Mini-Project 7 — TMA-Accelerated SDF Query (Hopper+)

Implement a signed-distance-field (SDF) lookup kernel that uses TMA to bulk-load 3D SDF grid tiles into shared memory for a batch of query points. Compare against a baseline with manual `cp.async` loads. If Hopper hardware is unavailable, implement the `cp.async` version targeting Ampere and write a design document describing how TMA would change the implementation.

Why this matters: SDF-based collision checking (used in cuRobo, CHOMP, TrajOpt) requires random 3D grid lookups with trilinear interpolation — a memory-bound pattern that TMA can accelerate via hardware address generation.

Week 8 — Triton and High-Level GPU DSLs

Goal: Explore alternatives to raw CUDA; understand when a DSL is the right tool for rapid kernel development.

Readings

1. Triton Language Documentation — Tutorial, language reference, and compilation model (tile-based programming, auto-tuning).
2. Modular Platform — Mojo language, MAX engine, and the portability argument.
3. Triton vs. Modular — Tradeoffs between embedded DSLs and standalone compilers.

Supplementary

- Triton source: `python/tutorials/` — Walk through the matmul and fused-softmax tutorials.

Mini-Project 8 — Triton Collision Kernel + Autotuning

Re-implement your Week 2 sphere–sphere collision kernel in Triton. Use Triton's `@triton.autotune` decorator to search over `BLOCK_SIZE_OBS`, `BLOCK_SIZE_CFG`, and `num_warps`. Compare performance against your hand-tuned CUDA kernel. Write a 1-page analysis of what Triton makes easier, what it makes harder, and where you hit expressiveness limits.

Why this matters: For rapid prototyping of new planning algorithms, a Triton kernel can be 5× faster to write than CUDA with 80% of the performance. Knowing when to use which tool is a key engineering skill.

Phase 3: Motion Planning on GPU (Weeks 9–12)

Week 9 — GPU-Accelerated Collision Checking at Scale

Goal: Study production GPU collision-checking systems and implement a high-performance collision pipeline.

Readings

1. Sundaralingam et al., "cuRobo: Parallelized Collision-Free Robot Motion Generation" (ICRA 2023) — The reference GPU motion planning system; focus on the swept-sphere collision model and batched kinematics.
2. J. Pan & D. Manocha, "GPU-based parallel collision detection for fast motion planning" (IJRR 2012) — Foundational paper on BVH-based parallel collision checking.
3. NVIDIA OptiX / Vulkan ray-tracing pipeline overview (high-level) — How RT cores can accelerate geometric queries.

Supplementary

- cuRobo source code: `curobo/geom/sdf/` and `curobo/cuda_robot_model/` — Study the CUDA kernels.

Mini-Project 9 — Batched Swept-Volume Collision Pipeline

Build a complete collision pipeline: given a robot represented as a chain of swept spheres (capsules), batch-evaluate M candidate trajectory segments against a set of environment primitives (spheres, capsules, boxes). Fuse FK computation, sphere placement, and collision distance into a single kernel launch (or CUDA graph). Target: evaluate 4096 trajectory segments of 32 timesteps each in < 1 ms on an RTX 4090.

Week 10 — Parallel Sampling-Based Planning

Goal: Understand how to parallelize inherently sequential tree/graph algorithms (RRT, RRT*, PRM) on GPUs.

Readings

1. Bialkowski et al., "Massively Parallelizing the RRT and the RRT*" (IROS 2011) — Early work on GPU-parallel RRT with batched nearest-neighbor and collision checks.
2. B. Ichter, E. Schmerling, & M. Pavone, "Group Marching Tree (GMT*)" (IJRR 2020) — A planning algorithm designed for GPU parallelism; wavefront expansion instead of sequential tree growth.
3. J. Thomason, Z. Kingston, & L. Kavraki, "Motions in Microseconds via Vectorized Sampling-Based Planning" (ICRA 2024) — Recent work on vectorized planning with SIMD/GPU acceleration.

Supplementary

- cuRobo's graph-planner module and trajectory optimization pipeline.

Mini-Project 10 — GPU-Parallel PRM Construction & Query

Implement a GPU-accelerated PRM pipeline: (a) generate N random C-space samples, (b) batch k-NN graph construction (reuse your Week 3 kernel), (c) parallel edge validation via batched collision checking, (d) shortest-path query using parallel BFS or Dijkstra (study Gunrock or use a simple parallel label-correcting approach). Benchmark end-to-end construction time for N = 50K–500K nodes in a 7-DOF C-space with 100 obstacles.

Week 11 — GPU Trajectory Optimization (MPPI / iLQR)

Goal: Study gradient-free and gradient-based trajectory optimization on GPU, which represent the state of the art in real-time motion planning.

Readings

1. G. Williams et al., "Model Predictive Path Integral Control Using Covariance Variable Importance Sampling" (IJRR 2018) — The MPPI algorithm; inherently GPU-parallel via batched rollout evaluation.
2. Bhardwaj et al., "STORM: An Integrated Framework for Fast Joint-Space Model-Predictive Control for Reactive Manipulation" (CoRL 2022) — GPU-MPPI with learned cost functions and warm-starting.
3. Plancher et al., "GPU-Accelerated iLQR" (relevant ICRA/RSS workshop papers) — Parallelizing the backward pass and batched forward simulation.

Supplementary

- NVIDIA Isaac Lab / Isaac Sim GPU simulation documentation.
- cuRobo's TrajOpt module source.

Mini-Project 11 — MPPI Controller with GPU Rollouts

Implement a complete MPPI controller for a 7-DOF arm reaching a target while avoiding obstacles: (a) sample K = 4096 action perturbations, (b) roll out dynamics (simple kinematic or second-order) on GPU, (c) evaluate collision + goal cost using your Week 9 pipeline, (d) compute importance-weighted mean. Run at ≥ 50 Hz control rate. Integrate with a simple simulator (PyBullet or MuJoCo via CPU, or Isaac Gym if available).

Week 12 — Capstone Integration & Future Directions

Goal: Combine everything into a complete GPU motion planning system; survey frontier topics.

Readings

1. Sundaralingam et al., cuRobo full paper — Re-read with fresh eyes; you now understand every kernel.
2. Le & Plaku, "Multi-GPU Motion Planning" or Deng et al., "Multi-GPU PRM" — Scaling to multiple GPUs.
3. Survey: "Learning-based motion planning" (any recent survey, e.g., Fishman et al.) — Where neural components (learned SDFs, neural collision checkers, diffusion planners) meet GPU compute.

Supplementary

- CXL/NVLink topology and multi-GPU memory models.
- FlashAttention paper (for the software-pipelining and tiling methodology, transferable to non-ML kernels).

Capstone Project — Real-Time GPU Motion Planner

Design and implement a complete real-time motion planning system that integrates at least three components you built during the course. Suggested architecture:

1. **Scene representation:** GPU-resident SDF grid (Week 7) or sphere decomposition, updated asynchronously via CUDA streams.
2. **Planning backbone:** Either a GPU-PRM (Week 10) for global planning or MPPI (Week 11) for local reactive planning — or a hybrid.
3. **Collision pipeline:** Fused FK + swept-sphere checking (Week 9) captured in a CUDA graph.
4. **Control loop:** 50+ Hz replanning with warm-started trajectories.

Deliverable: a 4-page technical report (IEEE format) describing the system architecture, kernel design decisions, and benchmarks (planning time, solution quality, GPU utilization). Include a roofline analysis positioning your key kernels.

Reading List — Quick Reference

Core CUDA Resources

#	Title	Topic	Week
1	Brief History of GPUs	GPU evolution	1
2	GPU Glossary	Terminology reference	1
3	Maximize Bandwidth & Latency (GTC)	Memory model	1, 2
4	Shared Memory Microbenchmarks	Bank conflicts	2
5	Fast Matrix Transpose on Hopper	Banking & layout	2
6	CUDA GEMM Step-by-Step	Tiling, optimization	3, 5
7	Register-Level Tiling	Data movement hierarchy	3
8	CUDA Graphs for GenAI	Launch overhead, graphs	4

#	Title	Topic	Week
9	Outperforming cuBLAS on H100	Production GEMM	5
10	CuTE Layouts	CUTLASS layout abstraction	5
11	Tensor Core Evolution	Hardware matrix units	6
12	WMMA/MMA Code + Notes	Tensor core programming	6
13	Fast GEMM with Tensor Cores	From-scratch TC GEMM	6
14	TMA with CuTE	Async bulk copy	7
15	Blackwell GEMM (CUTLASS)	Tensor Memory, 5th-gen TC	7
16	Persistent Dynamic Launch	Dynamic scheduling	7

DSL Resources

#	Title	Topic	Week
17	Triton Lang	Python GPU DSL	8
18	Modular Platform	Mojo / MAX	8
19	Triton vs. Modular	DSL comparison	8

Motion Planning Papers

#	Title	Topic	Week
20	cuRobo (ICRA 2023)	GPU motion planning system	9, 12
21	Pan & Manocha (IJRR 2012)	GPU BVH collision detection	9
22	Bialkowski et al. (IROS 2011)	Parallel RRT/RRT*	10
23	GMT* (IJRR 2020)	GPU-native planning algorithm	10
24	Thomason et al. (ICRA 2024)	Vectorized sampling-based planning	10
25	MPPI (IJRR 2018)	GPU trajectory optimization	11
26	STORM (CoRL 2022)	Reactive GPU-MPPI	11

Weekly Deliverables & Assessment

Each week, the student submits:

1. **Reading notes** (1 page): Key takeaways, one thing that surprised you, one question you still have.
2. **Working code**: Mini-project implementation with a README documenting build instructions, benchmarks, and a short analysis.
3. **Profiling artifacts**: At least one Nsight Systems timeline or Nsight Compute report per project showing you understand the performance characteristics of your kernel.

Grading weight (suggested): Reading notes 40%, Mini-projects 40%, Capstone 20%.

Suggested Meetings & Milestones

- **Weekly 1-on-1** (30 min): Discuss readings, review profiling results, debug kernel issues.
- **Week 4 checkpoint:** Student should be comfortable writing and profiling CUDA kernels; shared memory and streams should feel natural.
- **Week 8 checkpoint:** Student should have strong intuition for GPU performance limits (compute-bound vs. memory-bound), tiling strategies, and when to use CUDA vs. Triton.
- **Week 10:** Capstone proposal due (1-page description of planned system).
- **Week 12:** Capstone presentation (20 min + 10 min Q&A) and report submission.